

Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Organización de Lenguajes y Compiladores II.
Ing. Mario Tobar.



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

PIGLATIN - MANUAL DE USUARIO

Estudiante	Carné
201931811	Diego José Maldonado Monterroso

Quetzaltenango, Guatemala.

Lunes 24 de agosto de 2026.

REQUISITOS Y ARRANQUE

Software	Versión mínima	Comprobar con
JDK	21 o superior	java --version
Apache Maven	3.8 o superior	mvn --version

Desde la carpeta `codex_latinus` ([repositorio](#) previamente clonado):

```
mvn -o clean javafx:run
```

La primera vez, quita el `-o` (modo *offline*) para que Maven descargue las dependencias y genere el lexer y el parser desde los `.g4`. A partir de ahí `-o` arranca sin necesidad de descargar dependencias de nuevo.

Comando	Para qué
<code>mvn -o clean compile</code>	Solo compilar
<code>mvn -o clean package</code>	Generar el <code>.jar</code>
<code>mvn -o clean javafx:run</code>	Compilar y ejecutar

```
Problems Output Debug Console Terminal Ports GitHub
Diego Maldonado@DiegoPC MINGW64 ~/Desktop/Universidad 2026/SS 26/COMPI II/Practica_1/codex_latinus (main)
$ mvn -o clean javafx:run
[INFO] --- resources:3.3.1:resources (default-resources) @ codex_latinus ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] --- compiler:3.8.0:compile (default-compile) @ codex_latinus ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 68 source files to C:\Users\Diego Maldonado\Desktop\Universidad 2026\SS 26\COMPI II\Practica_1\codex_latinus\target\classes
[INFO] --- javafx:0.0.8:run (default-cli) < process-classes @ codex_latinus ---
[INFO] --- javafx:0.0.8:run (default-cli) @ codex_latinus ---
[INFO] A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by com.sun.glass.utils.NativeLibLoader in module javafx.graphics (file:/C:/Users/Diego%20Maldonado/.m2/repository/org/openjfx/javafx-graphics/21.0.9/javafx-graphics-21.0.9-win.jar)
WARNING: Use --enable-native-access-javafx.graphics to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

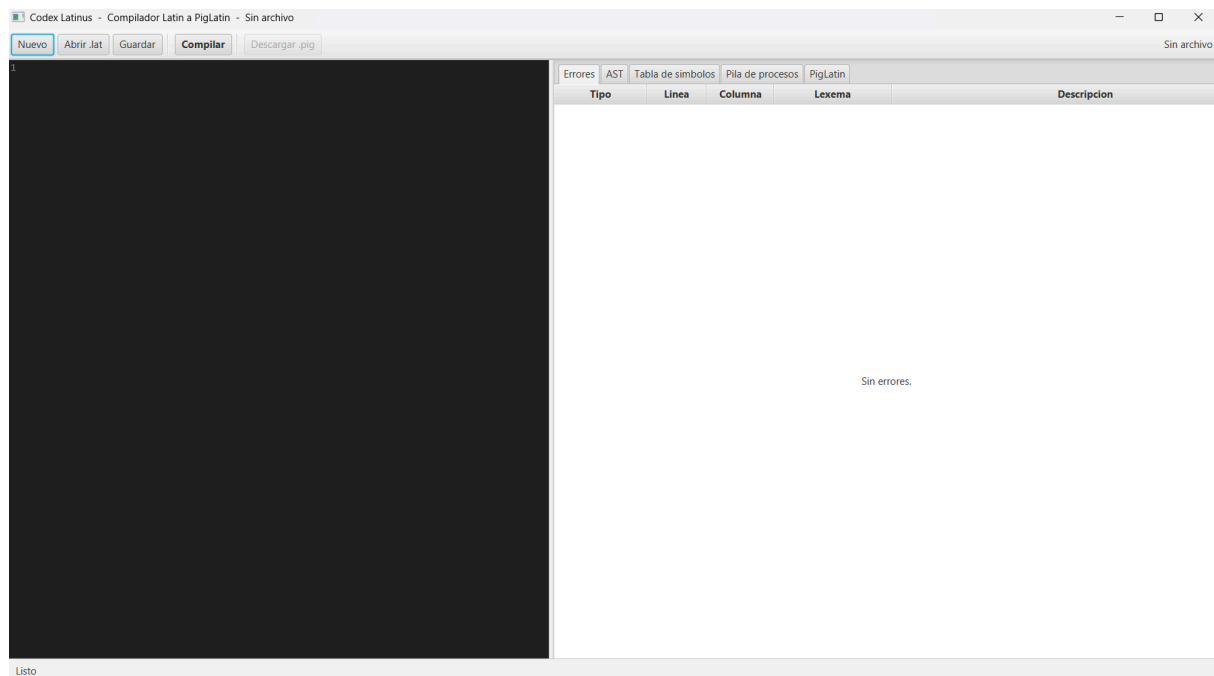
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::allocateMemory has been called by com.sun.marlin.OffHeapArray (file:/C:/Users/Diego%20Maldonado/.m2/repository/org/openjfx/javafx-graphics/21.0.9/javafx-graphics-21.0.9-win.jar)
WARNING: Please consider reporting this to the maintainers of class com.sun.marlin.OffHeapArray
WARNING: sun.misc.Unsafe::allocateMemory will be removed in a future release
[]
```

Ventana.

Tres zonas, de arriba abajo:

Zona	Qué contiene
Barra de herramientas	Los cinco botones y, a la derecha, el nombre del archivo actual
Centro	El editor a la izquierda y las cinco pestañas a la derecha, separados por una barra divisoria que se arrastra
Barra de estado	Una línea con el resultado de la última operación

Las cinco pestañas son las cinco salidas que pide el enunciado: **Errores**, **AST**, **Tabla de símbolos**, **Pila de procesos** y **PigLatin**.



Barra de herramientas.

Botón	Qué hace
Nuevo	Limpia el editor y olvida el archivo actual
Abrir .lat	Abre un archivo .lat y lo compila automáticamente
Guardar	Guarda. La primera vez pregunta dónde; después sobrescribe sin preguntar
Compilar	Analiza lo que hay en el editor y llena las cinco pestañas
Descargar .pig	Guarda la traducción. Deshabilitado hasta que una compilación salga limpia

A la derecha se ve el nombre del archivo actual, o «Sin archivo».

La barra de estado, abajo, resume la última operación:

Mensaje	Significa
Compilación exitosa N simbolo(s) N paso(s) de análisis	En verde. Todo bien
Compilación con N error(es) N paso(s) de análisis	En rojo. Revisa la pestaña Errores
Guardado en C:\...	Confirmación de guardado, con la ruta completa
Traducción descargada en C:\...	Confirmación de la descarga del .pig

Editor y su coloreado

El editor colorea **mientras escribes**, y lo hace con el **analizador léxico del propio compilador** — no con un plugin. Cada color corresponde a un tipo de token:

Color	Qué pinta
Naranja negrita	Palabras reservadas: esto, si, dum, reddere, VARIABLES>...
Verde negrita	Tipos: numerus, textum, bool, verum, falsus...
Verde claro	Números: 20, 9.81
Salmón	Textos y caracteres: "Resistencia", 'a'
Verde <i>cursiva</i>	Comentarios: // ... y /* ... */
Rojo subrayado	Lo que el lenguaje no reconoce

El rojo subrayado cubre cuatro casos, y aparece **antes de compilar**, en cuanto terminas de teclear el carácter:

Caso	Ejemplo
Símbolo desconocido	esto # : numerus 1;
Cadena sin cerrar	esto n : textum "sin cerrar;
Carácter sin cerrar	esto c : littera 'a;
Comentario de bloque sin cerrar	/* nunca cierra

Codex Latinus - Compilador Latin a PigLatin - resistencia.lat

NuevoAbrir.latGuardarCompilarDescargar.pig

resistencia.lat

5678910111213141516171819202122232425262728293031323334353637383940414243444546

```
VARIABLES>
esto edad : numerus 20;
esto nombre : textum "Resistencia";
esto gravedad : decimalis 9.81;
esto inicial : littera 'a';
esto activo : bool verum;
esto bloqueado : falsus;

series numeros[2] : numerus {1, 1};
series nombres[2] : textum {"Hola", "Adios"};

structura Persona {
    esto nombre : textum;
    esto edad : numerus;
} finis;

esto lider : Persona { nombre: "Ana", edad: 34 }

MUNERA>

actio atacarCerdos(esto fuerza : numerus) {
    >> "Atacando con fuerza: " >> fuerza;
} finis;

ratio numerus calcularPoder(esto fuerza : numerus, esto nivel : numerus)
VARIABLES[
    esto total : numerus 0;
]
per (esto i : numerus 0; i < nivel; i++) {
    total = total + fuerza;
} finis;
reddere total;
} finis;

ratio textum saludar(esto quien : textum) {
    si (quien == "cerdo") {
        reddere "Oink";
    } aliter (quien == "humano") {
        reddere "Salve, " + quien;
    } aliter /

```

ErroresASTTabla de símbolosPila de procesosPigLatin

☐ Traducir contenido de los textos

ARIABLESVAY>
estoway edadway : umerusnay 20;
estoway ombrenay : extumtay "Resistencia";
estoway avedadgray : ecimalisday 9.81;
estoway inicialway : itteralay 'a';
estoway activoway : oolbay erumvay;
estoway oqueadoblay : alsufay;
eriessay umerosnay[2] : umerusnay {1, 1};
eriessay ombresnay[2] : extumtay {"Hola", "Adios"};
ucturastray Ersonapay {
 estoway ombrenay : extumtay;
 estoway edadway : umerusnay;
} inisfay;
estoway iderlay : Ersonapay { ombrenay: "Ana", edadway: 34 };

UNERAMAY>
actioway atacarCerdosway(estoway uerzafay : umerusnay) {
 %OINK "Atacando con fuerza: " %OINK uerzafay;
} inisfay;
atioway umerusnay alcularPoderay(estoway uerzafay : umerusnay, estoway ivelnay : umerusnay) {
 ARIABLESVAY[
 estoway oaltay : umerusnay 0;
]
 erpay (estoway iway : umerusnay 0; iway < ivelnay; iway++) {
 otaltay = otaltay + uerzafay;
 } inisfay;
 eddereray otaltay;
} inisfay;
atioway extumtay aludarsay(estoway uienqay : extumtay) {
 isay (uienqay == "cerdo") {
 eddereray "Oink";
 } aliterway (uienqay == "humano") {
 eddereray "Salve, " + uienqay;
 } aliterway {

Compilacion exitosa | 22 simbolo(s) | 1858 paso(s) de analisis

Codex Latinus - Compilador Latin a PigLatin - lexico_invalido.lat

NuevoAbrir.latGuardarCompilarDescargar.pig

lexico_invalido.lat

12345678910111213141516

```
// Espera: errores LEXICOS. El pipeline se detiene sin intentar parsear.
// Cuatro clases distintas, cada una con su propio mensaje.
VARIABLES>
esto edad : numerus 20;
esto f : numerus 1; // simbolo no reconocido
esto saludo : textum "sin cerrar;
esto inicial : littera 'a';
MAIOR>
>> edad;
/* este comentario nunca cierra
FINIS;

```

ErroresASTTabla de símbolosPila de procesosPigLatin

Tipo	Linea	Columna	Lexema	Descripción
Lexico	6	6	#	Simbolo no reconocido por el lenguaje.
Lexico	7	22	"sin cerrar;	Cadena sin cerrar: falta la comilla doble final.
Lexico	8	24	'a	Caracter sin cerrar: falta la comilla simple final.
Lexico	14	1	/* este comentario nun... FINIS;	Comentario de bloque sin cerrar: falta "*/".

Compilacion con 4 error(es) | 0 paso(s) de analisis

Pestaña Errores.

Columna	Qué muestra
Tipo	Lexico, Sintactico y Semantico
Línea / Columna	Posición exacta, empezando en 1
Lexema	El texto que causó el problema
Descripción	Qué pasó, en español

Doble clic en una fila mueve el cursor del editor hasta ese error. Es la forma más rápida de corregir un archivo con varios.

Los errores salen **ordenados por posición**, no por orden de detección: se leen de arriba abajo igual que el archivo. Clic en cualquier cabecera reordena la tabla.

Nunca verás errores léxicos y sintácticos a la vez. El compilador corta en el primer nivel que falla, porque todo lo posterior sería ruido derivado:

Si ves...	Significa
Solo léxicos	Hay algo que el lenguaje no reconoce. Ni se intentó parsear
Solo sintácticos	Los tokens son válidos pero el orden no. No se construyó el AST
Semánticos	La sintaxis está bien; el problema es de tipos, ámbitos o reglas

The screenshot displays the Codex Latinus compiler window. The top menu bar includes options like 'Nuevo', 'Abrir .lat', 'Guardar', 'Compilar', and 'Descargar pig'. Below the menu is a toolbar with icons for file operations. The main editor area shows a Latin program snippet:

```
1 // Espera: textum solo admite '+', y bool no participa en aritmetica.  
2 VARIABLES>  
3  
4 esto edad : numerus 20;  
5 esto nombre : textum "Ana";  
6 esto activo : bool verum;  
7  
8 MAIOR>  
9  
10 >> nombre - edad;  
11 >> nombre * 2;  
12 >> activo + edad;  
13 >> non edad;  
14  
15 FINIS;  
16
```

To the right of the editor is a panel titled 'tipos_operacion.lat' which contains a table summarizing errors:

Tipo	Línea	Columna	Lexema	Descripción
Semantico	10	4	-	Operacion invalida: 'textum' - 'numerus'. Recuerde que textum solo admite...
Semantico	11	4	*	Operacion invalida: 'textum' * 'numerus'. Recuerde que textum solo admite...
Semantico	12	4	+	Operacion invalida: 'bool' + 'numerus'.
Semantico	13	4	non	El operador 'non' no puede aplicarse a 'numerus'.

At the bottom left, a status bar indicates: 'Compilación con 4 error(es) | 247 paso(s) de analisis'.

AST.

El árbol sintáctico abstracto del programa, **ya expandido**.

- Se llena **solo si el programa es válido**. Con errores queda vacío a propósito: el enunciado exige graficar el AST únicamente cuando el lenguaje es correcto.
- Cada nodo muestra una etiqueta legible: PROGRAMA, esto edad : numerus, SI, DUM, +...
- Las flechas de la izquierda pliegan y despliegan cada rama.

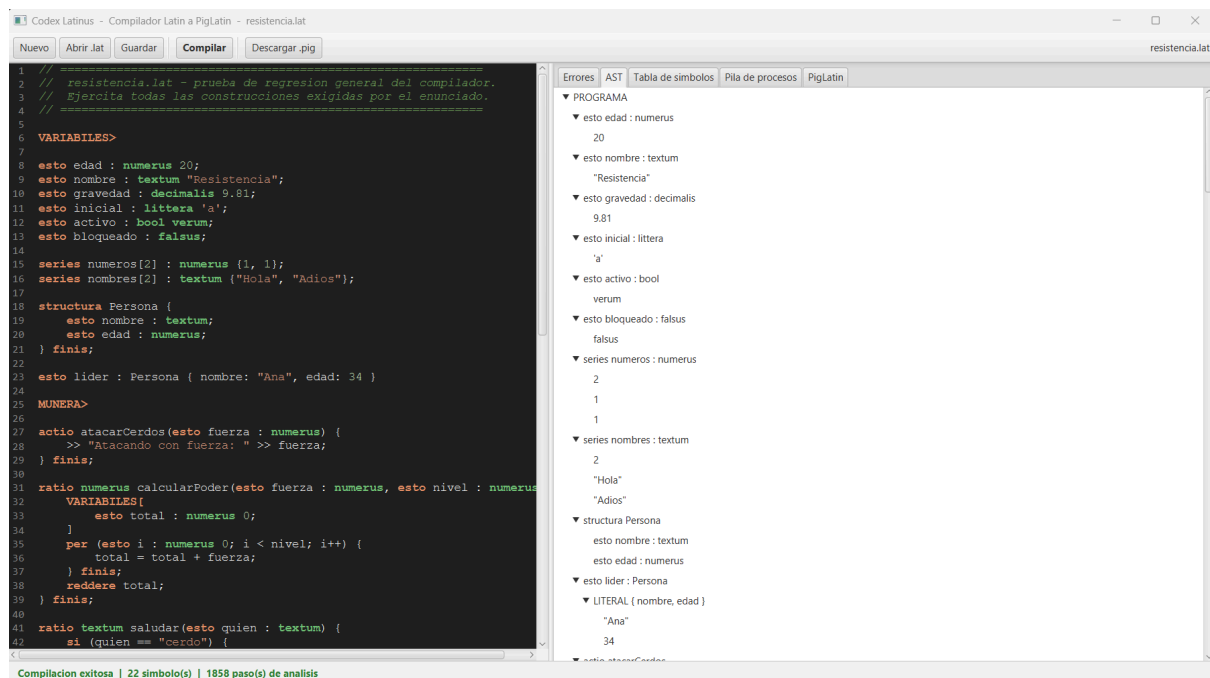


Tabla de símbolos.

Todo lo declarado en el programa, de todos los ámbitos.

Columna	Qué muestra
Nombre	El identificador
Categoría	Variable, Parámetro, Atributo, Arreglo, Función o Estructura
Tipo	Como lo escribiste: numerus, Persona...
Ámbito	global, MAIOR, el nombre de la función, o per_1, per_2...
Línea / Columna	Dónde se declaró
Detalle	Tamaño de un arreglo, firma de una función, atributos de una estructura

Muestra **todos** los símbolos, incluso los locales de funciones que ya terminaron. Ordena por **Ámbito** haciendo clic en la cabecera para agruparlos por función.

The screenshot shows the Codex Latinus IDE with a PigLatin program and its execution results. The program defines variables, a Person structure, and functions for attacking and saluting. The execution results table shows the state of variables and function calls.

Codex Latinus - Compilador Latin a PigLatin - resistencia.lat

Nuevo **Abrir .lat** **Guardar** **Compilar** **Descargar .pig**

```

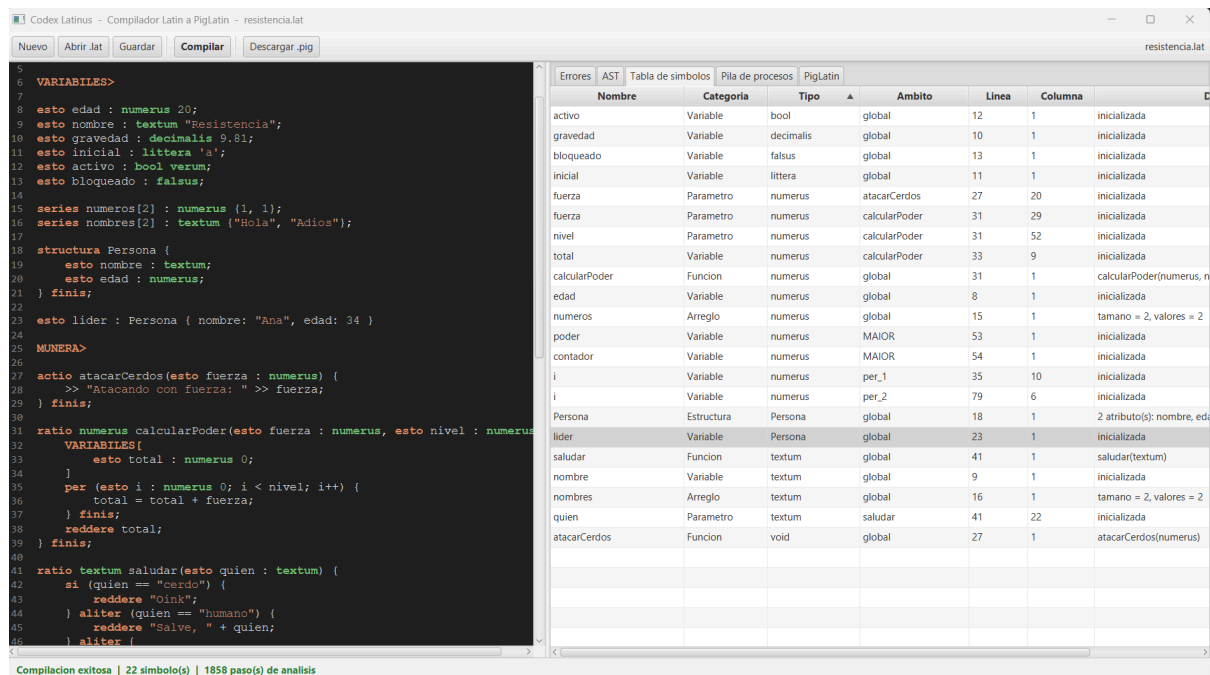
5  VARIABLES>
6
7
8  esto edad : numerus 20;
9  esto nombre : textum "Resistencia";
10 esto gravedad : decimalis 9.81;
11 esto inicial : littera 'a';
12 esto activo : bool verum;
13 esto bloqueado : falsus;
14
15 series numeros[2] : numerus {1, 1};
16 series nombres[2] : textum {"Hola", "Adios"};
17
18 estructura Persona {
19     esto nombre : textum;
20     esto edad : numerus;
21 } finis;
22
23 esto lider : Persona { nombre: "Ana", edad: 34 }
24
25 MUNERA>
26
27 actio atacarCerdos(esto fuerza : numerus) {
28     >> "Atacando con fuerza: " >> fuerza;
29 } finis;
30
31 ratio numeros calcularPoder(esto fuerza : numerus, esto nivel : numerus)
32 VARIABLES{
33     esto total : numerus 0;
34 }
35 per (esto i : numerus 0; i < nivel; i++) {
36     total = total + fuerza;
37 } finis;
38 reddere total;
39 } finis;
40
41 ratio textum saludar(esto quien : textum) {
42     si (quien == "cerdo") {
43         reddere "Oink";
44     } aliter (quien == "humano") {
45         reddere "Salve, " + quien;
46     } aliter {

```

Errores **AST** **Tabla de simbolos** **Pila de procesos** **PigLatin**

Nombre	Categoria	Tipo	Ambito	Linea	Columna	
fuerza	Parametro	numerus	atacarCerdos	27	20	inicializada
fuerza	Parametro	numerus	calcularPoder	31	29	inicializada
nivel	Parametro	numerus	calcularPoder	31	52	inicializada
total	Variable	numerus	calcularPoder	33	9	inicializada
Persona	Estructura	Persona	global	18	1	2 atributo(s): nombre, edad
atacarCerdos	Funcion	void	atacarCerdos(numerus)	27	1	atacarCerdos(numerus)
calcularPoder	Funcion	numerus	global	31	1	calcularPoder(numerus, r
saludar	Funcion	textum	global	41	1	saludar(textum)
edad	Variable	numerus	global	8	1	inicializada
nombre	Variable	textum	global	9	1	inicializada
gravedad	Variable	decimalis	global	10	1	inicializada
inicial	Variable	littera	global	11	1	inicializada
activo	Variable	bool	global	12	1	inicializada
bloqueado	Variable	falsus	global	13	1	inicializada
numeros	Arreglo	numerus	global	15	1	tamano = 2, valores = 2
nombres	Arreglo	textum	global	16	1	tamano = 2, valores = 2
lider	Variable	Persona	global	23	1	inicializada
poder	Variable	numerus	MAIOR	53	1	inicializada
contador	Variable	numerus	MAIOR	54	1	inicializada
i	Variable	numerus	per_1	35	10	inicializada
i	Variable	numerus	per_2	79	6	inicializada
quien	Parametro	textum	saludar	41	22	inicializado

Compilador exitoso: 133 simbolos(s), 14588 proces(s) de analisis



Pila de procesos.

Reproduce el análisis sintáctico **paso a paso**. Arriba van los botones **< Anterior** y **Siguiente >** y el contador Paso N de M; debajo, dos paneles:

- **Izquierda — Pila de llamadas:** las reglas gramaticales activas en ese momento, de la cima a la base. La cima está marcada con >.
- **Derecha — Log de procesos:** las operaciones desde el paso 1 hasta el paso actual.

Ir y venir es seguro: el log se **reconstruye** en cada movimiento, no se acumula, así que nunca aparecen líneas duplicadas.

Operación	Significa
PUSH (regla activa)	Se activó una regla gramatical
SHIFT	Se consumió un token de la entrada
REDUCE / REPLACE	La regla terminó y se reemplazó por su nodo
ACCEPT	La cadena fue aceptada por la gramática
ERROR	Token inesperado

La pila se muestra aunque el archivo tenga errores. Es a propósito: un archivo que no compila es justamente aquel cuyos pasos ERROR conviene leer.

The screenshot shows the Codex Latinus compiler interface. The main window displays a Latin program with various declarations and functions. The right-hand panel shows the execution log, which lists the steps of the compilation process, including the activation of rules and the consumption of tokens.

Program Code (resistencia.lat):

```

5
6  VARIABILES>
7
8  esto edad : numerus 20;
9  esto nombre : textum "Resistencia";
10 esto gravedad : decimalis 9.81;
11 esto inicial : littera 'a';
12 esto activo : bool verum;
13 esto bloqueado : falsus;
14
15 series numeros[2] : numerus {1, 1};
16 series nombres[2] : textum ("Hola", "Adios");
17
18 structura Persona {
19     esto nombre : textum;
20     esto edad : numerus;
21 } finis;
22
23 esto lider : Persona { nombre: "Ana", edad: 34 }
24
25 MINERA>
26
27 actio atacarCerdos(esto fuerza : numerus) {
28     >> "Atacando con fuerza: " >> fuerza;
29 } finis;
30
31 ratio numerus calcularPoder(esto fuerza : numerus, esto nivel :
32     VARIABILES[
33         esto total : numerus 0;
34     ]
35     per (esto i : numerus 0; i < nivel; i++) {
36         total = total + fuerza;
37     } finis;
38     reddere total;
39 } finis;
40
41 ratio textum saludar(esto quien : textum) {
42     si (quien == "cerdo") {
43         reddere "Oink";
44     } aliter (quien == "humano") {
45         reddere "Salve, " + quien;
46     } aliter {

```

Execution Log (Log de procesos):

- [1] PUSH (regla activa) (6:1) Se activa la regla <programa>
- [2] PUSH (regla activa) (6:1) Se activa la regla <seccionVariables>
- [3] SHIFT (6:1) Se consume el token VARIABILES -> 'VARIABILES'
- [4] PUSH (regla activa) (8:1) Se activa la regla <declaracionGlobal>
- [5] PUSH (regla activa) (8:1) Se activa la regla <declaracionVariable>
- [6] SHIFT (8:1) Se consume el token ESTO -> 'esto'
- [7] SHIFT (8:6) Se consume el token ID -> 'edad'
- [8] SHIFT (8:11) Se consume el token DOS_PUNTOS -> ':'
- [9] PUSH (regla activa) (8:13) Se activa la regla <tipo>
- [10] SHIFT (8:13) Se consume el token NUMERUS -> 'numerus'
- [11] REDUCE / REPLACE (8:13) Se reduce <tipo> a un nodo del arbol
- [12] PUSH (regla activa) (8:21) Se activa la regla <expresion>

At the bottom, a status bar indicates: "Compilacion exitosa | 22 simbolo(s) | 1858 paso(s) de analisis".

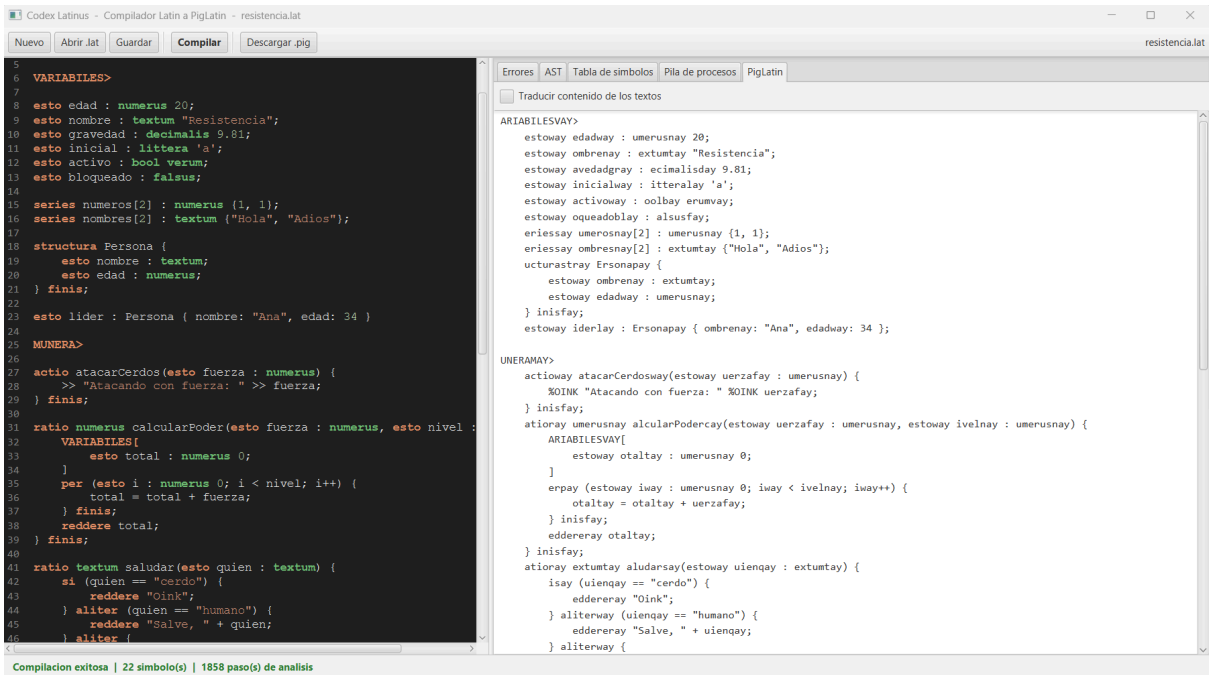
PigLatin.

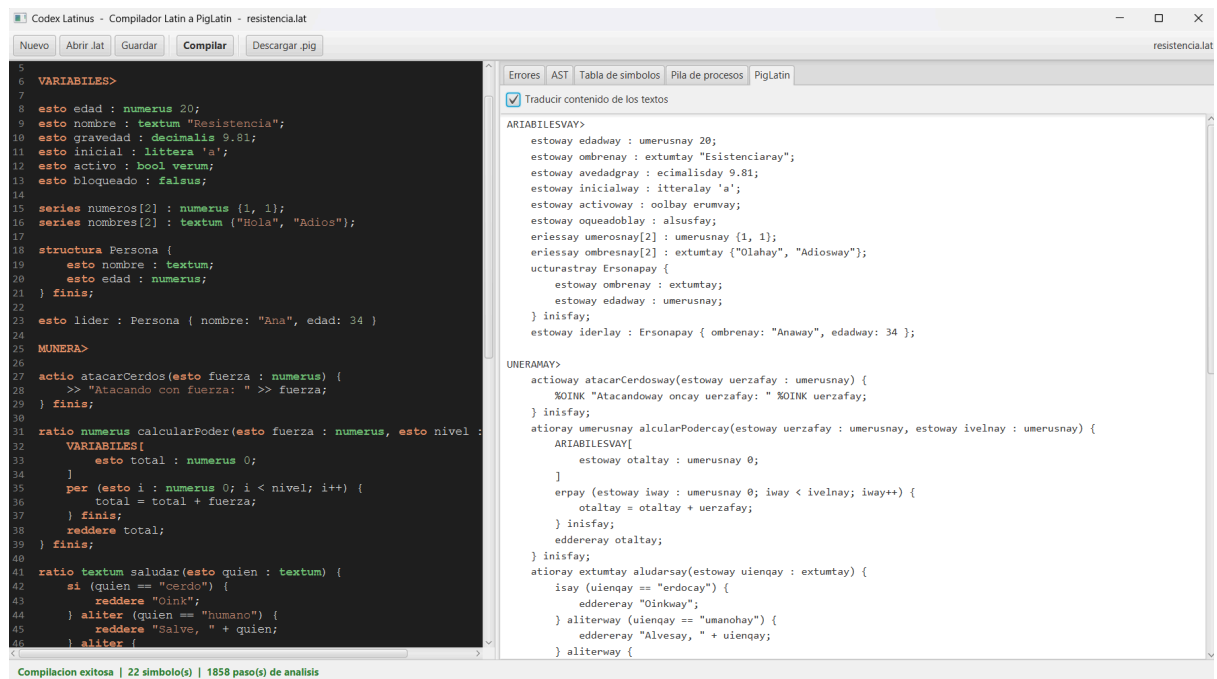
La traducción del programa, y una casilla:

Casilla	Por defecto	Qué hace
Traducir contenido de los textos	desmarcada	Traduce también lo que va dentro de "..."

Marcarla o desmarcarla **retraduce al instante**, sin recompilar.

Las palabras reservadas, los identificadores y los marcadores de sección se traducen siempre. La única elección real es el interior de un textum, porque el enunciado no obliga a traducirlo.





Escribir un programa .lat

Lo mínimo que compila:

```

MAIOR>
    >> "Hola comandante";
FINIS;

```

La estructura completa son tres secciones y el cierre:

```
VARIABLES>           // opcional: variables, arreglos y estructuras globales
    esto edad : numerus 20;

MUNERA>               // opcional: solo definiciones de funciones
    ratio numerus doble(esto n : numerus) {
        VARIABLES[
            esto resultado : numerus 0;
        ]
        resultado = n * 2;
        reddere resultado;
    } finis;

MAIOR>                // OBLIGATORIA: la funcion principal
    >> doble(edad);

FINIS;
```

Cuatro reglas que conviene tener a mano:

Regla	Detalle
Es <i>case sensitive</i>	FINIS; cierra el programa ; finis; cierra un bloque
Solo MAIOR> es obligatoria	VARIABLES> y MUNERA> pueden faltar
Las variables de una función van al principio	Dentro de VARIABLES[...], no a media función ni dentro de un si
La lectura no lleva ;	edad << es la única instrucción sin punto y coma